

✓ UPI Transaction Analysis — Data Cleaning

Objective

This notebook demonstrates a step-by-step process for cleaning a noisy UPI transaction dataset.

The cleaning process focuses on:

- inconsistent column names
- irrelevant fields
- NULL and missing values
- unwanted spaces
- inconsistent categorical values
- incorrect amount formatting
- inconsistent timestamps
- incorrect derived time fields
- inconsistent fraud values

The cleaning process **does not remove transaction records**. Each major operation checks that the record count remains unchanged.

The original noisy CSV is loaded directly using `read_csv()`.

✓ Step 1 — Import the noisy dataset

We first load the noisy CSV file.

At this stage, the dataset is only imported. No rows are filtered, deleted, or dropped.

```
import pandas as pd
import numpy as np

# Enter the path of the noisy CSV file here.
file_path = "UPI_Transactions_2024.csv"

# Read the complete CSV.
# No on_bad_lines='skip' is used because records must not be silently discarded
df = pd.read_csv(
    file_path,
    dtype="string",
    engine="python"
```

```
)

# Store the actual number of records loaded.
# We do not assume any fixed number of rows.
starting_rows = len(df)

print("Dataset loaded successfully.")
print("Rows loaded:", starting_rows)
print("Columns loaded:", len(df.columns))
```

```
Dataset loaded successfully.
Rows loaded: 250000
Columns loaded: 27
```

✓ Step 2 — Inspect the noisy dataset

We inspect the original column names and sample records.

This helps identify formatting problems before starting the cleaning process.

```
print("Original column names:")
for i, column in enumerate(df.columns, start=1):
    print(i, ":", repr(column))

print("\nFirst 5 records:")
display(df.head())

print("\nLast 5 records:")
display(df.tail())

print("\nMissing values currently detected:")
display(
    df.isna().sum()
    .sort_values(ascending=False)
    .to_frame("Missing Values")
)

# Record-preservation check
assert len(df) == starting_rows
```


Original column names:
1 : ' Transaction ID '
2 : 'time_stamp'
3 : 'Transaction_Type'
4 : ' merchant category '
5 : 'amount_INR'
6 : 'Transaction Status'
7 : 'sender_agegroup'
8 : 'receiver_age_group '
9 : 'Sender State'
10 : 'sender bank'
11 : 'Receiver_Bank'
12 : 'device type'
13 : 'Network Type '
14 : 'Fraud Flag'
15 : 'hour of day'
16 : 'Day Of Week'
17 : 'is weekend'
18 : 'battery level %'
19 : 'App Version '
20 : 'device storage free gb'
21 : 'screen brightness %'
22 : 'Device Language'
23 : 'GPS accuracy m'
24 : 'nearby wifi count'
25 : 'session duration sec'
26 : 'notifications last hour'
27 : 'merchant distance km'

First 5 records:

	Transaction ID	time_stamp	Transaction_Type	merchant category	amount_INR	Transaction Status
0	TXN00000000001	10/8/2024 15:17	P2P	Entertainment	868	SUCCESS
1	TXN00000000002	2024/04/11 06:56:00	P2M	Grocery	1011	SUCCESS
2	TXN00000000003	4/2/2024 13:27	P2P	Grocery	477	SUCCESS
3	TXN00000000004	1/7/2024 10:09	P2P	Fuel	2784	SUCCESS
4	TXN00000000005	1/23/2024 19:04	P2P	Shopping	990	SUCCESS

5 rows × 27 columns

Last 5 records:

	Transaction ID	time_stamp	Transaction_Type	merchant category	amount_INR	Transaction Status
5	TXN00000000006	10/8/2024 15:17	P2P	Entertainment	868	SUCCESS
6	TXN00000000007	2024/04/11 06:56:00	P2M	Grocery	1011	SUCCESS
7	TXN00000000008	4/2/2024 13:27	P2P	Grocery	477	SUCCESS
8	TXN00000000009	1/7/2024 10:09	P2P	Fuel	2784	SUCCESS

249995	TXN0000249996	11/8/2024 22:41	Recharge	Food	373
249996	TXN0000249997	12/15/2024 2:58	P2P	Utilities	2025
249997	TXN0000249998	11/27/2024 16:33	P2P	Food	468
249998	TXN0000249999	1/5/2024 13:31	Recharge	Healthcare	284
249999	TXN0000250000	1/17/2024 15:23	P2P	Entertainment	531

5 rows × 27 columns

Missing values currently detected:

Missing Values	
battery level %	6301
screen brightness %	5098
notifications last hour	5088
nearby wifi count	5087
session duration sec	5083
merchant distance km	5082
device storage free gb	5081
GPS accuracy m	5074
merchant category	5064
Transaction_Type	5061
Transaction Status	5041
Device Language	5036
App Version	5034
time_stamp	4643
amount_INR	4636
device type	4585
Network Type	4578
receiver_age_group	4572
Sender State	4572
sender bank	4566
sender_agegroup	4563

✓ Step 3 — Standardize column names

Receiver Bank 4557

Fraud Flag 4144

Day Of Week 4079

is weekend 4077

hour of day 4045

Transaction ID 0

The noisy dataset contains inconsistent spaces, capitalization, and separators in its column names.

We standardize these names so that each field can be handled consistently during cleaning.

Only column labels are changed. Transaction records are not removed.

```
# First standardize the physical column names.
df.columns = (
    df.columns
    .astype(str)
    .str.strip()
    .str.lower()
    .str.replace("%", "", regex=False)
    .str.replace("(", "", regex=False)
    .str.replace(")", "", regex=False)
    .str.replace(r"^[a-z0-9]+", "_", regex=True)
    .str.strip("_")
)

# Convert standardized names into readable project names.
column_rename = {
    "transaction_id": "transaction id",
    "timestamp": "timestamp",
    "time_stamp": "timestamp",
    "transaction_type": "transaction type",
    "merchant_category": "merchant_category",
    "amount_inr": "amount (INR)",
    "transaction_status": "transaction_status",
    "sender_agegroup": "sender_age_group",
    "sender_age_group": "sender_age_group",
    "receiver_age_group": "receiver_age_group",
    "sender_state": "sender_state",
    "sender_bank": "sender_bank",
    "receiver_bank": "receiver_bank",
    "device_type": "device_type",
    "network_type": "network_type",
    "fraud_flag": "fraud_flag",
    "hour_of_day": "hour_of_day",
    "day_of_week": "day_of_week",
    "is_weekend": "is_weekend",
    "battery_level": "battery_level_pct",
    "app_version": "app_version",
    "device_storage_free_gb": "device_storage_free_gb",
    "screen_brightness": "screen_brightness_pct",
    "device language": "device language",
```

```

    "gps_accuracy_m": "gps_accuracy_m",
    "nearby_wifi_count": "nearby_wifi_count",
    "session_duration_sec": "session_duration_sec",
    "notifications_last_hour": "notifications_last_hour",
    "merchant_distance_km": "merchant_distance_km"
}

df.rename(columns=column_rename, inplace=True)

print("Column names standardized successfully.")
print(df.columns.tolist())

# Renaming columns cannot remove records.
assert len(df) == starting_rows

```

```

Column names standardized successfully.
['transaction id', 'timestamp', 'transaction type', 'merchant_category', 'amount

```

✓ Step 4 — Remove irrelevant columns

Some fields describe device, application, or environmental information that is not required for the UPI transaction analysis.

These fields are removed because they are outside the scope of the transaction analysis.

Only columns are removed. Transaction records remain unchanged.

```

irrelevant_columns = [
    "battery_level_pct",
    "app_version",
    "device_storage_free_gb",
    "screen_brightness_pct",
    "device_language",
    "gps_accuracy_m",
    "nearby_wifi_count",
    "session_duration_sec",
    "notifications_last_hour",
    "merchant_distance_km"
]

# Remove only columns that actually exist.
columns_to_remove = [
    column for column in irrelevant_columns
    if column in df.columns
]

```

```
df.drop(columns=columns_to_remove, inplace=True)
```

```
print("Irrelevant columns removed.")  
print("Removed columns:", columns_to_remove)
```

```
# Dropping columns must never change the record count.  
assert len(df) == starting_rows
```

```
Irrelevant columns removed.  
Removed columns: ['battery_level_pct', 'app_version', 'device_storage_free_gb',
```

✓ Step 5 — Standardize NULL and missing values

Missing information may appear as blank text, `NULL`, `N/A`, `NA`, or `None`.

These different representations are converted into one standard missing-value representation so that they can be handled consistently.

Rows are not removed.

```
missing_markers = [  
    "",  
    " ",  
    "NULL",  
    "null",  
    "N/A",  
    "NA",  
    "None",  
    "none"  
]  
  
for column in df.columns:  
    df[column] = df[column].replace(missing_markers, np.nan)  
  
print("NULL and missing-value representations standardized.")  
  
assert len(df) == starting_rows
```

```
NULL and missing-value representations standardized.
```

✓ Step 6 — Remove unnecessary spaces

Text values may contain leading spaces, trailing spaces, or repeated spaces.

These differences can make identical values appear as separate categories, so unnecessary spaces are removed.

```
text_columns = [
    "transaction type",
    "merchant_category",
    "transaction_status",
    "sender_age_group",
    "receiver_age_group",
    "sender_state",
    "sender_bank",
    "receiver_bank",
    "device_type",
    "network_type"
]

for column in text_columns:
    if column in df.columns:
        df[column] = (
            df[column]
            .astype("string")
            .str.strip()
            .str.replace(r"\s+", " ", regex=True)
        )

print("Unnecessary spaces cleaned.")

assert len(df) == starting_rows
```

Unnecessary spaces cleaned.

✓ Step 7 — Correct obvious categorical errors

Some categorical values contain obvious spelling or formatting mistakes.

These values are corrected so that the same transaction category is represented consistently.

```
if "transaction_status" in df.columns:
    df["transaction_status"] = df["transaction_status"].replac
        "SUCESS": "SUCCESS",
        "Sucess": "SUCCESS",
        "sucess": "SUCCESS",
        "FAILD": "FAILED",
        "Faild": "FAILED",
        "faild": "FAILED",
        "PENDNG": "PENDING",
```

```

        "Pendng": "PENDING",
        "pendng": "PENDING"
    })

if "transaction type" in df.columns:
    df["transaction type"] = df["transaction type"].replace({
        "P-2-P": "P2P",
        "p-2-p": "P2P",
        "P-2-M": "P2M",
        "p-2-m": "P2M"
    })

print("Obvious categorical errors corrected.")

assert len(df) == starting_rows

```

Obvious categorical errors corrected.

✓ Step 8 — Standardize merchant categories

Merchant category values may use inconsistent capitalization or spacing.

We standardize their representation so that the same category is not treated as different categories because of formatting.

```

if "merchant_category" in df.columns:
    df["merchant_category"] = (
        df["merchant_category"]
        .astype("string")
        .str.strip()
        .str.lower()
        .str.replace(r"\s+", " ", regex=True)
    )

print("Merchant categories standardized.")

assert len(df) == starting_rows

```

Merchant categories standardized.

✓ Step 9 — Clean the transaction amount

The transaction amount can contain currency symbols, **INR**, commas, spaces, or other unwanted characters.

These formatting characters are removed and the field is converted to numeric form.

Values that cannot be converted become missing values and are handled later without deleting their transaction records.

```
if "amount (INR)" in df.columns:
    amount = df["amount (INR)"].astype("string")

    amount = amount.str.strip()
    amount = amount.str.replace("₹", "", regex=False)
    amount = amount.str.replace("INR", "", regex=False)
    amount = amount.str.replace(",", "", regex=False)
    amount = amount.str.replace(r"\s+", "", regex=True)

    df["amount (INR)"] = pd.to_numeric(
        amount,
        errors="coerce"
    )

print("Transaction amount cleaned.")

assert len(df) == starting_rows
```

Transaction amount cleaned.

✓ Step 10 — Standardize the timestamp

The timestamp is required for analysing transaction timing.

Different timestamp formats can occur in noisy data, so the values are converted into a consistent datetime format.

Values that cannot be interpreted are treated as missing instead of deleting the transaction.

```
if "timestamp" in df.columns:
    df["timestamp"] = pd.to_datetime(
        df["timestamp"],
        errors="coerce",
        format="mixed"
    )

print("Timestamp standardized.")

assert len(df) == starting_rows
```

Timestamp standardized.

✓ Step 11 — Recalculate time-related fields

The hour, day of week, and weekend fields are derived from the timestamp.

Because noisy values may contain incorrect derived information, these fields are calculated again from the cleaned timestamp.

```
if "timestamp" in df.columns:
    df["hour_of_day"] = df["timestamp"].dt.hour
    df["day_of_week"] = df["timestamp"].dt.day_name()
    df["is_weekend"] = (
        df["timestamp"].dt.dayofweek.isin([5, 6])
    ).astype("Int64")

print("Time-related fields recalculated.")

assert len(df) == starting_rows
```

Time-related fields recalculated.

✓ Step 12 — Standardize the fraud indicator

The fraud indicator can contain different representations such as `TRUE`, `FALSE`, `1`, and `0`.

These representations are converted into one consistent binary numeric format.

```
if "fraud_flag" in df.columns:
    fraud = (
        df["fraud_flag"]
        .astype("string")
        .str.strip()
        .str.upper()
    )

    fraud = fraud.replace({
        "TRUE": "1",
        "FALSE": "0"
    })

    df["fraud_flag"] = pd.to_numeric(
        fraud,
        errors="coerce"
```

```
)

print("Fraud indicator standardized.")

assert len(df) == starting_rows
```

```
Fraud indicator standardized.
```

✓ Step 13 — Handle remaining missing values

Some values may still be missing after the earlier corrections.

Instead of removing those transaction records, missing values are filled using appropriate values from the existing dataset.

- Numerical transaction amounts use the median.
- Categorical fields use their most frequent valid value.
- Missing timestamps use the most frequent valid timestamp when available.

This keeps the existing transaction records in the dataset.

```
# -----
# STEP 13 – HANDLE ALL REMAINING MISSING VALUES
# -----

# 1. Clean and fill missing transaction amounts
if "amount (INR)" in df.columns:

    # Make sure amount is numeric
    df["amount (INR)"] = pd.to_numeric(
        df["amount (INR)"],
        errors="coerce"
    )

    # Fill missing amounts with the median
    median_amount = df["amount (INR)"].median()

    if pd.notna(median_amount):
        df["amount (INR)"] = df["amount (INR)"].fillna(
            median_amount
        )

# 2. Fill missing categorical values
categorical_columns = [
    "transaction type",
    "merchant_category",
    "transaction status",
```

```

    "sender_age_group",
    "receiver_age_group",
    "sender_state",
    "sender_bank",
    "receiver_bank",
    "device_type",
    "network_type",
    "fraud_flag"
]

for column in categorical_columns:

    if column in df.columns:

        # Find the most common valid value
        mode_values = df[column].dropna().mode()

        if len(mode_values) > 0:
            most_common_value = mode_values.iloc[0]

            df[column] = df[column].fillna(
                most_common_value
            )

# 3. Handle missing timestamp values
if "timestamp" in df.columns:

    # Find the most common valid timestamp
    timestamp_values = df["timestamp"].dropna().mode()

    if len(timestamp_values) > 0:
        most_common_timestamp = timestamp_values.iloc[0]

        df["timestamp"] = df["timestamp"].fillna(
            most_common_timestamp
        )

# 4. Recalculate fields that depend on timestamp
if "timestamp" in df.columns:

    df["hour_of_day"] = df["timestamp"].dt.hour

    df["day_of_week"] = df["timestamp"].dt.day_name()

    df["is_weekend"] = (
        df["timestamp"].dt.dayofweek.isin([5, 6])
    ).astype("Int64")

# 5. Final safety check for any remaining missing values

```

```

# 5. Final safety check for any remaining missing values
# If a column still contains missing values, fill it using
# its most frequent valid value.

for column in df.columns:

    if df[column].isna().any():

        # Try the most frequent value first
        mode_values = df[column].dropna().mode()

        if len(mode_values) > 0:
            df[column] = df[column].fillna(
                mode_values.iloc[0]
            )

        # If the entire column is somehow missing,
        # use a suitable fallback.
        else:

            if pd.api.types.is_numeric_dtype(df[column]):
                df[column] = df[column].fillna(0)

            else:
                df[column] = df[column].fillna("Unknown")

# -----
# FINAL VALIDATION
# -----

remaining_missing = int(df.isna().sum().sum())

print("Total missing values after cleaning:", remaining_missing)

print("Rows after cleaning:", len(df))

# Make sure NO transaction records were removed.
assert len(df) == starting_rows, (
    "ERROR: The number of records changed during cleaning."
)

# Make sure NO missing values remain.
assert remaining_missing == 0, (
    "ERROR: Missing values are still present."
)

print()
print("SUCCESS: All missing values have been handled.")
print("SUCCESS: All original transaction records are preserved")
print("Total records:", len(df))

```

```
Total missing values after cleaning: 0  
Rows after cleaning: 250000
```

```
SUCCESS: All missing values have been handled.  
SUCCESS: All original transaction records are preserved.  
Total records: 250000
```

✓ Step 14 — Inspect the cleaned values

We now inspect the dataset after the cleaning operations.

This step checks remaining missing values and important fields. It does not remove records.

```
print("Current dataset shape:", df.shape)  
  
print("\nTotal missing values:")  
print(df.isna().sum().sum())  
  
print("\nMissing values by column:")  
display(  
    df.isna().sum()  
    .to_frame("Missing Values")  
)  
  
assert df.isna().sum().sum() == 0  
  
print("\nNo missing values remain.")
```

Current dataset shape: (250000, 17)

Total missing values:
0

Missing values by column:

	Missing Values	
transaction id	0	
timestamp	0	
transaction type	0	
merchant_category	0	
amount (INR)	0	
transaction_status	0	
sender_age_group	0	
receiver_age_group	0	
sender_state	0	
sender_bank	0	
receiver_bank	0	
device_type	0	
network_type	0	
fraud_flag	0	
hour_of_day	0	
day_of_week	0	
is_weekend	0	

No missing values remain.

✓ Step 15 — Verify record preservation

The cleaning process should correct the existing information without deleting transaction records.

We compare the record count at the beginning and end.

The notebook uses the actual number loaded from the CSV, so it does not assume any particular number of records.

```

final_rows = len(df)

print("Rows before cleaning:", starting_rows)
print("Rows after cleaning :", final_rows)

if final_rows != starting_rows:
    raise ValueError(
        "Record count changed during cleaning. "
        "Check the previous cells for an unintended row-removal operation."
    )

print("SUCCESS: All transaction records are preserved.")

```

```

Rows before cleaning: 250000
Rows after cleaning : 250000
SUCCESS: All transaction records are preserved.

```

✓ Step 16 — Save the cleaned dataset

The cleaned data is saved as a separate CSV file.

The original noisy CSV is not overwritten.

```

output_path = "upi_transactions_2024_CLEANED.csv"

df.to_csv(
    output_path,
    index=False
)

print("Cleaned dataset saved successfully.")
print("File:", output_path)

```

```

Cleaned dataset saved successfully.
File: upi_transactions_2024_CLEANED.csv

```

✓ Step 17 — Final inspection

A small sample of the cleaned dataset is displayed for a final visual check before starting the UPI Transaction Analysis.

```

display(df.head())

print("\nFinal dataset information:")

```

df.info()

	transaction id	timestamp	transaction type	merchant_category	amount (INR)	transactio
0	TXN0000000001	2024-10-08 15:17:00	P2P	entertainment	868.0	S
1	TXN0000000002	2024-04-11 06:56:00	P2M	grocery	1011.0	S
2	TXN0000000003	2024-04-02 13:27:00	P2P	grocery	477.0	S
3	TXN0000000004	2024-01-07 10:09:00	P2P	fuel	2784.0	S
4	TXN0000000005	2024-01-23 19:04:00	P2P	shopping	990.0	S

Final dataset information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 250000 entries, 0 to 249999
Data columns (total 17 columns):
Column Non-Null Count Dtype
--- -----
0 transaction id 250000 non-null string
1 timestamp 250000 non-null datetime64[ns]
2 transaction type 250000 non-null string
3 merchant_category 250000 non-null string
4 amount (INR) 250000 non-null Float64
5 transaction_status 250000 non-null string
6 sender_age_group 250000 non-null string
7 receiver_age_group 250000 non-null string
8 sender_state 250000 non-null string
9 sender_bank 250000 non-null string
10 receiver_bank 250000 non-null string
11 device_type 250000 non-null string
12 network_type 250000 non-null string
13 fraud_flag 250000 non-null Int64
14 hour_of_day 250000 non-null int32
15 day_of_week 250000 non-null object
16 is_weekend 250000 non-null Int64
dtypes: Float64(1), Int64(2), datetime64[ns](1), int32(1), object(1), string(11)
memory usage: 32.2+ MB



Conclusion

The noisy UPI transaction dataset has been cleaned through a sequence of data-quality operations.

The cleaned CSV is now ready for the next stage of the project:

UPI Transaction Analysis

Double-click (or enter) to edit

Double-click (or enter) to edit